

第28回 実践：Web ゲームプログラミング7（ゲームオーバーとリスタート）

- 前回：得点・残機を jQuery で表示した
- 今回：残機 0 で GAME OVER にし、ボタンから再開できるようにする
- 次回：全体を確認して修正する

実習時のフォルダ構成

Web_名前 の中に 28 フォルダを作る。第 27 回の `sample` フォルダと `exercise` フォルダを、28 の中にコピーして、続きから作る。

```
Web_名前
├── 28
│   ├── sample          ← 第 27 回の sample をコピーする
│   │   ├── index.html
│   │   ├── style.css
│   │   └── main.js
│   └── exercise        ← 第 27 回の exercise をコピーする
│       ├── index.html
│       ├── style.css
│       └── main.js
```

- `sample`：講義で使うサンプル（講義内容のコードは、`sample/main.js` に書き足す）
- `exercise`：自分のシューティングゲーム（演習 1～2、発展）
- 今回は、`index.html` にゲームオーバーのパネルを、`style.css` にパネルの設定を追加する。

講義内容

1. 前回の復習

- `$("#score").text(score);` のように、jQuery で HTML の表示を書き換えた。
- `checkPlayerHit`：敵が自機にぶつかったら、敵を消して、残機を 1 減らし、表示を書き換える。

今回は、残機が 0 になったらゲームを止めて、HTML のゲームオーバーのパネルを表示する。パネルのリスタートボタンを押すと、最初からやり直せるようにする。これで、シューティングゲームの基本が完成する。

2. 今回の main.js の並び

```
// ===== 準備 =====  
// ===== ゲームの状態 ===== ← gameState を追加する  
// ===== 自機・弾・敵 =====  
// ===== キー入力 =====  
// ===== リスタート ===== ← 今回追加する (resetGame と、ボタンのクリック)  
// ===== 更新 ===== ← checkPlayerHit を書き換える  
// ===== 描画 =====  
// ===== ゲームループ ===== ← gameLoop を書き換える
```

3. ゲームオーバーのパネル (HTML と CSS)

HTML と CSS は、次のとおりに書き写す。今回の中心は、JavaScript の部分。

sample/index.html の得点の表示欄の下に、パネルを追加する。

```
<!DOCTYPE html>  
<html lang="ja">  
<head>  
  <meta charset="UTF-8">  
  <title>シューティングゲーム</title>  
  <link rel="stylesheet" href="style.css">  
</head>  
<body>  
  <p>SCORE: <span id="score">0</span> LIVES: <span id="lives">3</span></p>  
  <div id="gameOverPanel">  
    <p>GAME OVER</p>  
    <p>最終得点: <span id="finalScore">0</span></p>  
    <button id="restartButton">リスタート</button>  
  </div>  
  <canvas id="game" width="400" height="500"></canvas>  
  <script src="https://code.jquery.com/jquery-4.0.0.min.js" integrity="sha384-  
    fgGyf7Mo7DURSOMnOy7ed+dkq5Job205Gnzu6Qlg0B0HKaq4D76Dt8VIDCzcMHV" crossorigin="anonymous"></script>  
  <script src="main.js"></script>  
</body>  
</html>
```

sample/style.css に、パネルの設定を追加する。

```
body {  
  margin: 0;  
  overflow: hidden;  
}  
  
#gameOverPanel {  
  display: none;  
  padding: 8px;  
  background-color: #eeeeee;  
}
```

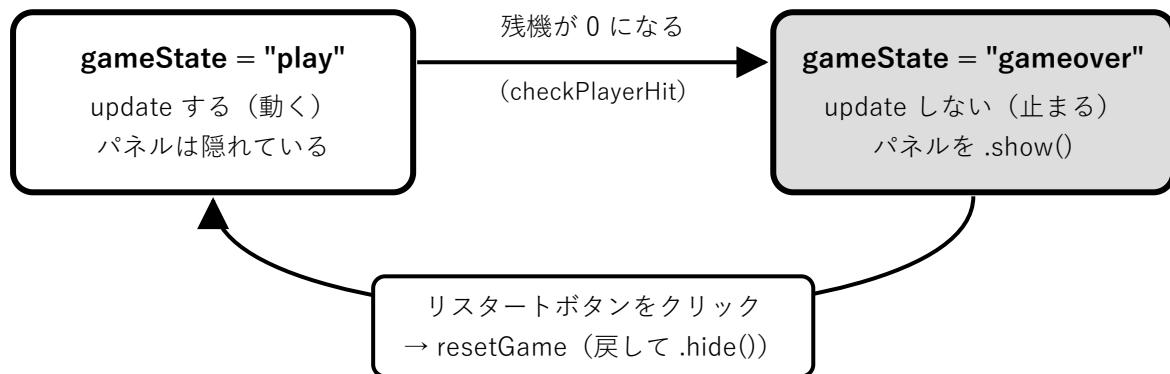
- `<div id="gameOverPanel">` : GAME OVER の文字、最終得点、リスタートボタンを 1 つにまとめたパネル。

- `display: none;` : その要素を表示しない、という CSS。パネルは、ゲームの最初は見えないようにしておく。
- GAME OVER は Canvas には描かず、HTML のパネルで表示する。

4. ゲームの状態 (gameState)

ゲームには、「プレイ中」「ゲームオーバー」のような「状態」がある。今の状態を変数 `gameState` に入れておき、状態によって行う処理を変える。今回作るのは、次の図の流れ。

draw (描く) は、どちらの状態でも毎フレーム行う



「ゲームの状態」の部分の先頭に追加する。

```
// ===== ゲームの状態 =====
let gameState = "play";           // "play": プレイ中  "gameover": ゲームオーバー
let score = 0;
let lives = 3;
let frameCount = 0;               // ゲーム開始からのフレーム数
```

「ゲームループ」の部分の `gameLoop` を、次のように書き換える。

```
// ===== ゲームループ =====
function gameLoop() {
  if (gameState === "play") {
    update();
  }
  draw();
  requestAnimationFrame(gameLoop);
}
```

- `gameState` が `"play"` のときだけ、`update` を呼び出す。ゲームオーバーになると、自機も弾も敵も止まる。
- `draw` と `requestAnimationFrame` は、状態に関係なく続ける。ゲームループそのものは止めない。

5. 残機 0 で GAME OVER にする (.show)

「更新」の部分の `checkPlayerHit` を、次のように書き換える (`if (lives <= 0) { ... }` の部分を追加する)。

```
// 敵と自機の当たり判定
function checkPlayerHit() {
  for (let i = enemies.length - 1; i >= 0; i--) {
    if (isHit(player, enemies[i])) {
      enemies.splice(i, 1);
      lives -= 1;
      $("#lives").text(lives);
      if (lives <= 0) {
        gameState = "gameover";
        $("#finalScore").text(score);
        $("#gameOverPanel").show();
      }
      break; // 1 フレームで減る残機は 1 つまで
    }
  }
}
```

- 残機が 0 以下になったら、`gameState` を "gameover" にして、ゲームを止める。
- `$("#finalScore").text(score);` : パネルの最終得点に、今の得点を入れる (第 27 回の `.text()`)。
- `$("#gameOverPanel").show();` : パネルを表示する。

jQuery	すること
<code>\$("#id").show();</code>	要素を表示する
<code>\$("#id").hide();</code>	要素を隠す

- `.show()` と `.hide()` は、対になる操作。今回は、ゲームオーバーで `.show()`、リスタートで `.hide()` を使う。

6. リスタート（resetGame と .on("click")）

リスタートするには、ゲームの途中で変わった変数を、最初の値に戻す。HTML の表示も戻し、パネルを隠す。「キー入力」の部分の下に、「リスタート」の部分を追加する。

```
// ===== リスタート =====
function resetGame() {
  gameState = "play";
  score = 0;
  lives = 3;
  frameCount = 0;
  player.x = 184;
  bullets = [];
  enemies = [];
  shotTimer = 0;
  $("#score").text(score);
  $("#lives").text(lives);
  $("#gameOverPanel").hide();
}

$("#restartButton").on("click", resetGame);
```

- `$("#score")...` より上の部分：変数を最初の値に戻す。`bullets = [];`・`enemies = [];` は、空の配列を代入し直して、画面の弾と敵をすべて消す（`bullets` と `enemies` を `let` で作ったのは、このため）。
- `$("#score").text(score);`・`$("#lives").text(lives);`：変数を戻しただけでは、HTML の表示は古いまま。表示も戻す。
- `$("#gameOverPanel").hide();`：パネルを隠す。
 - パネルを隠すと、リスタートボタンも一緒に隠れる。ボタンが選ばれた状態のまま残って、スペースキーを押したときにボタンがもう一度押される、ということも防げる。
- `$("#restartButton").on("click", resetGame);`：リスタートボタンがクリックされたら、`resetGame` を呼び出す。
 - 第 20 回の `scoreButton.addEventListener("click", addTenPoints);` と同じ考え方。jQuery では `.on("click", 関数名)` と書く。
 - 関数名に `()` を付けない（関数そのものを渡す）のも、第 20 回と同じ。
- 最初の値は、ファイルの上の方と `resetGame` の 2 か所を書いてある。値を変えるときは、両方を変える。
- 自分のゲームで、ほかにも途中で変わる値（自機の y 座標や速さなど）を作った場合は、それも `resetGame` で最初の値に戻す。

7. うまく動かないとき

症状	確認すること
最初からパネルが表示されている	<code>style.css</code> の <code>#gameOverPanel { display: none; }</code> 。 <code>index.html</code> で <code>style.css</code> を読み込んでいるか
残機が 0 でも止まらない	<code>gameLoop</code> の <code>if (gameState === "play")</code> 。 <code>checkPlayerHit</code> で <code>gameState = "gameover";</code> にしているか
パネルが出ない	<code>\$("#gameOverPanel")</code> の綴りと <code>#</code> 。HTML の <code>id</code> と同じ名前か
ボタンを押しても何も起きない	<code>\$("#restartButton").on("click", resetGame);</code> を書いたか。 <code>resetGame()</code> と <code>()</code> を付けていないか
リスタート後に敵や弾が残る、表示が戻らない	<code>resetGame</code> で戻し忘れていた変数・表示がないか（次回、全体を確認する）

演習

`exercise/main.js`（自分のシューティングゲーム）に書く。

演習 1 ゲームオーバーのパネルを出す

- `exercise/index.html` と `exercise/style.css` に、講義 3 と同じパネルと設定を追加する。
- `gameState` を追加し、`gameLoop` を書き換える。
- `checkPlayerHit` を書き換えて、残機が 0 になったらゲームが止まり、パネルに最終得点が表示されるようにする。

演習 2 リスタートボタンで再開する

- 次の空欄（`_____`）を埋めて、`resetGame` を完成させる。戻す値は、自分のゲームの最初の値にする。

```
function resetGame() {  
  gameState = "play";  
  score = _____;  
  lives = _____;  
  frameCount = 0;  
  player.x = _____;  
  bullets = _____;  
  enemies = _____;  
  shotTimer = 0;  
  $("#score").text(score);  
  $("#lives").text(lives);  
  $("#gameOverPanel")._____;  
}
```

- `.on("click", resetGame)` で、リスタートボタンから再開できるようにする。

- 「残機 0 → GAME OVER → リスタートボタン → 再開」が、一度できればよい。戻し忘れの細かい確認は、次回行う。

発展（任意）

- ゲームオーバーのとき、Enter キーでもリスタートできるようにする。（ヒント：`keydown` の中で、`event.key === "Enter"` かつ `gameState === "gameover"` のときに `resetGame()` を呼ぶ。）